

Version Control for Code You Did Not Write

One edit per function, filed under the request that asked for it

Anonymous Author(s)

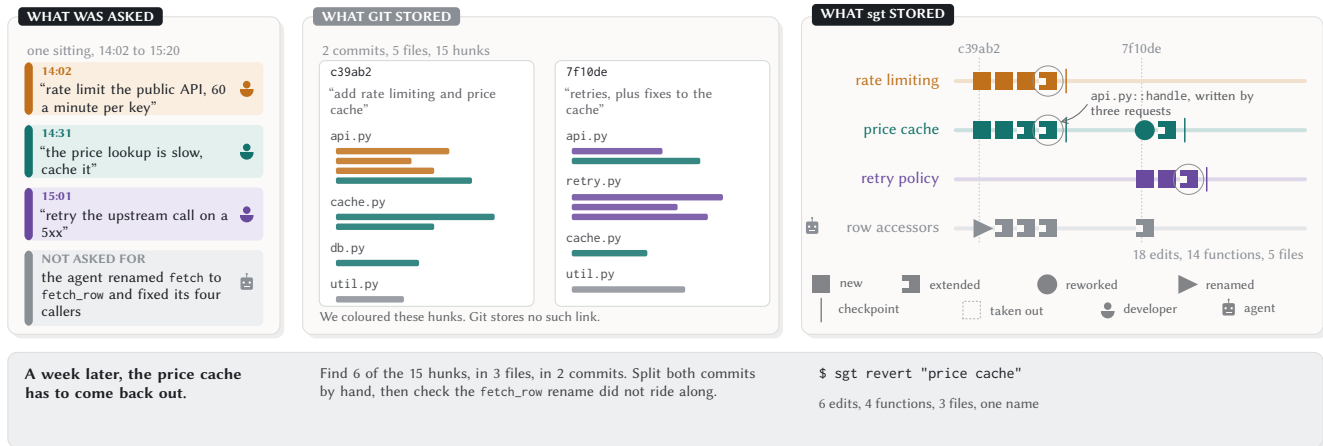


Figure 1: One hour with a coding agent, recorded two ways. The developer asked for three things and the agent did a fourth nobody asked for, and the two records differ in whether anything in the repository says which is which.

Abstract

A developer who works through a coding agent describes a change in English and receives code, so the description that defines the change and the lines that implement it are written by different parties. One developer asked for three things in an hour and fourteen functions in five files came back changed, in two commits that hold all three requests interleaved with a rename the agent made on its own initiative. Git addresses a history by lines and by commits, so a developer who wants one of those three requests out of it has to name the lines that request produced, and after an agent wrote the code the developer cannot name them. The unit of record and the unit of thought have come apart. A history should record what its author asked for and treat the resulting text as the secondary product, which is the reverse of what every system in wide use does.

We present sgt, a version control system that records a change as one edit to one function, files each edit under the request it was made for, and keeps the whole record inside an ordinary git repository. A developer names a request to remove it, put it back, or carry it to another branch, and the code that other requests contributed to the same files stays where it is. sgt reads code and does not write it, with one exception it reports every time: removing a request from a function that later work also changed subtracts the request's lines from the current text, and the developer's tests decide whether that was right.

The design costs the developer three things. Two edits to the same function from the same starting point are recorded as competing versions that a person decides between, including the pair at opposite ends of a fifty line function that git would have merged without comment. The grouping of edits into named features is inferred from which functions change together, so a person corrects

it. Code in a language sgt does not parse is versioned whole, at the grain git already uses.

sgt has versioned its own construction for fifty-seven days and three hundred and forty-six commits, and that is where the limits we report come from: a quarter of the repository is not code, fifty-six files no set of records reproduces, and a safety gate nobody configured. The study of whether a developer gets these operations right more often with sgt than without it is designed and not yet run, and we state which of its measures would count against us before we have seen any of them.

Keywords

version control, coding agents, program comprehension, developer tools, change history

1 Introduction

When a developer works through a coding agent, the change they want and the code that implements it become two separate descriptions, and the developer authors only the first one. A developer on one of our projects asked for a price lookup to be cached because it had become slow, and the agent added a cache module, gave the database query a parameter for how long a cached price stays good, changed the request handler to consult the cache before the database, and chose the names of the three new functions. Over the following hour that developer made two more requests, and while carrying out the third the agent revised two of the functions it had written for the second. Barke et al. describe programmers in this mode accepting what comes back after a glance that confirms its shape matches what they asked for [4], and among 410 developers surveyed by Liang et al., understanding and trusting generated code was one of the most frequently reported difficulties [48]. The three

sentences the developer typed stayed in the chat window, and what entered the repository was eighteen changes to fourteen functions in five files.

A week later that developer wants the caching gone, and the request has to be restated in whatever terms their version control system understands. Those terms are lines and commits, in every system in wide use. Git records a version as a snapshot of the whole working tree [74], Darcs and Pijul record it as a patch that adds and removes lines in a stated context [52, 63], and under either design the smallest thing a history can name is a line in a file, so `git revert` takes back one commit, `git cherry-pick` moves one commit, and a branch points at one commit. The caching is six changes to four functions in three files, spread across the two commits that hour produced, and neither commit holds only those six. Commits that carry work for more than one purpose are ordinary. Herzig and Zeller found them throughout the histories of five open source Java projects [31, 35], and the tools built since to separate them are scored on whether they recover a grouping a person supplied by hand [5, 17, 61, 66, 69].

Separating the caching out by hand means deciding, for each of the fifteen hunks in those two commits, whether it belongs to the caching, to the rate limiting the developer asked for half an hour before the caching, to the retry policy they asked for half an hour after it, or to a rename the agent performed without being asked. Six of the fifteen turn out to be the caching, and the developer then splits both commits so those six stand alone, reverses them, and checks that no function they kept calls a function they just deleted. Biggerstaff et al. called the task of recovering which functions implement a named concern the concept assignment problem [6], and the feature location literature that followed measures how much of a program a developer has to read before they can answer one such question [19, 51]. Xia et al. instrumented professional developers and found that roughly 58% of their working time went to understanding code [83].

For as long as developers wrote the lines themselves this recovery was cheap. The person doing it remembered writing the code, so the history had only to remind them of a decomposition they already held [46, 68]. Version control systems that record at the grain of a method were built during those years and several are still available, and almost nobody adopted one, which we read as evidence that the grain was never the constraint that mattered [13, 30, 36]. Naur argued that a program text and its documentation are insufficient carriers of the theory a programmer builds while writing it, and that the continued modification of a large program therefore depends on a group of programmers in close and continuous contact with it [56]. A developer who directs an agent stands outside that group from the first minute. They did not write the lines, they remember nothing about them, and the theory Naur describes was held, for as long as it was held at all, by something that discards it when the session ends.

What that developer does hold is the sentence they typed, which named the boundary of the change exactly, in their own words, before any of the code existed, and which no version control system records. A history, we think, should be regarded as a record of what its author asked for, with the text those requests produced as the secondary product. Getting this unit wrong does not put an operation out of a developer's reach. It makes them pay for the

operation in reading, every time they want it, and it teaches them to call ordinary difficulty what is a property of their notation.

A careful maintainer will answer all of this with commit hygiene: commit small, commit often, stage hunk by hunk with `git add -p`, and each request ends up with a commit of its own, a discipline the community has studied carefully enough to have opinions about what a good commit message contains [73]. We have asked contributors on our own projects for exactly this and have held ourselves to it, and it works whenever the developer knows at the moment of committing which lines serve which purpose. In an agent session the developer does not know that. When fourteen changed functions come back and the developer's own contribution was three sentences of English, staging hunk by hunk means reading the whole diff to reconstruct a boundary the agent held and never wrote down. Thimbleby argued that a notation should let a person defer a decision until they hold the information the decision needs [72]. Commit hygiene asks for the decision first, which was affordable for twenty years because the information arrived with the authorship.

We present `sgt`, a version control system that records a change as one edit to one function and files each edit under the request it was made for. The developer keeps the editor and the agent they already use, and `sgt` reads what those two leave in the files, so nothing about how the developer works has to change. A save turns the pending changes into one record per changed function, and each record names the function it changed, the version it started from, the version it produced, and the other functions it was built on. `sgt` writes those records into the repository and commits them with the code, so a colleague who has not installed `sgt` pulls an ordinary git history. Figure 1 shows the hour described above under both records.

Recording at that grain gives each of the operations the developer already wanted a command whose argument is a name. `sgt revert "price cache"` removes the six edits filed under the price cache and names every function where later work had built on them, and the rate limiting in the same file is byte for byte what it was. `sgt show cache.py::get_cached` reports which request that function came from, which save reworked it, and how many edits would come out along with it. When the developer decides a week later that the cache was needed after all, `sgt restore` puts the same six edits back, and a database index a colleague added in the meantime is still there, because it was never in the set that came out. None of these commands asks the developer to look at a hunk.

This design costs the developer three things, all three of which they meet in the first hour rather than in some later edge case. Two edits to the same function made from the same starting point are recorded as two competing versions and a person has to decide which one survives, including the case of two edits at opposite ends of a fifty line function that touch no common line and that git would merge without comment. The grouping of edits into named requests is inferred from which functions change together and which call each other, so it is a guess and it will sometimes be wrong, and correcting it costs the developer a rename, a merge, or a split, all three of which move labels and leave every recorded edit alone. `sgt` reads Python, TypeScript, and TSX with `tree-sitter` [9] and versions every other file whole, at the grain git already uses, so

a repository whose interesting logic is written in Go gets nothing from sgt today.

This paper makes four contributions. We characterise the mismatch between the unit a developer using an agent decides in and the unit a version control system records, one attempted operation at a time (Section 3), following the style of conceptual analysis Pérez De Rosso and Jackson applied to git itself [58, 59]. We describe sgt’s design as four commitments, say what each one buys and what it costs, and run the whole set of operations on one repository with the reasoning behind each command (Sections 4 and 5). We state which observation would show each of our claims to be wrong, and fix the design of the study that would produce it before we hold its results (Section 6). We report what fifty-seven days of sgt tracking its own development has taught us about where the design breaks down, which is the one part of this paper whose numbers are real today (Section 7).

2 Related Work

2.1 Programming by request

Studies of programming with a language model are four years old, and the unit of work they measure changed inside that span. Barke et al. watched 20 programmers and found them using the model in two ways, finishing a line they had already planned, or asking for a starting point in code they did not yet know how to write [4]. Vaithilingam et al. found that 24 participants using Copilot did not complete tasks faster and still preferred to keep it on, because it gave them a draft to work from [78], and Sarkar et al. collected what the experience felt like from the developer’s side [64]. In both studies the developer accepted a line or a block of code. Four years later SWE-bench gives a model a natural language issue report from a real repository and scores it on whether the repository’s own tests pass afterwards [40], and the systems built to score well on it edit several files in a loop [84]. The unit a developer hands over has gone from a line to a paragraph of English, and the code that comes back is correspondingly larger.

All of this work studies the minutes around acceptance, asking whether the code is correct [60], whether the developer can tell that it is correct [79], whether they trust it [11, 48], and how much faster they finish [16, 55], and it stops once the developer has said yes. A week later that same developer wants to remove the caching they asked for and keep the rate limiting they asked for in the same hour, and removing one and keeping the other is a version control operation. We know of no study of what happens to agent-written code after it is committed. Acceptance is where this literature’s outcome measures end, so none of them can say what the code costs to own, and the cost of owning it is what a version control system is for.

2.2 What a version control system agrees to record

Git records a version as a snapshot of the whole working tree, and each commit names its parents plus a tree of file contents addressed by hash [74]. Verifying that a history has not been altered is then a matter of rehashing it, and a merge is computed on demand from a common ancestor, so nothing in the repository has to say what happened when two edits met. We keep the first of those properties

and give up the second on purpose. sgt addresses its records by their contents, so checking that a record has not been altered is still a matter of rehashing it; but when two edits to one function descend from the same parent, sgt records that they did rather than computing what the pair amounts to, and Section 4.3 says why a system whose unit is a function cannot afford to derive that answer. Pérez De Rosso and Jackson analysed git’s concepts against the purposes users bring to them and found several places where a concept serves a purpose users do not have, the staging area and detached HEAD among them. Their redesign, Gitless, removes both [38, 58, 59]. The commit survives their redesign untouched, and it should, because it serves a purpose users do have. The commit is the smallest thing git can address, so any operation over less than a commit works by having the developer point at lines, and git add -p and git rebase -i are the tools for pointing. Under git’s original constraints this is the cheapest correct design available, because the person running git add -p in 2005 had typed the lines minutes earlier and writing their decomposition into the repository would have meant maintaining a second structure that could disagree with the first.

Darcs takes the other option and records a version as a patch that adds and removes lines in a stated context [63]. Patches that do not depend on each other commute, so a user can pull the third patch in a series without the first two, and the theory behind this has been formalised more than once [39, 54]. Pijul reimplements it on a data structure designed to make the operations fast [52]. A Darcs user who pulls one patch onto another branch moves the patch that was recorded, along with the dependencies it names. Git’s cherry-pick reapplies a diff in a context it was not written against and produces a second commit with a second hash, so the same work has two identities and a later rebase has to be told which of them the developer meant. Darcs works out what a patch depends on from the lines the patch touches, so a dependency that leaves no trace in the text goes unrecorded. When an agent adds a Retry class in a new file and edits api.py:handle to call it, the two patches share no line, so nothing in the record says that pulling the second without the first yields a file referring to a name that does not exist. Darcs is explicit about this and offers a way to declare a dependency by hand, and we would have made the same call in 2003, because computing a dependency between two functions requires parsing the language and a system that parses works for some languages and not for others. Darcs declined that price and we pay it.

Jujutsu and Mercurial both add a record of history’s own history. Jujutsu gives each change an identifier separate from its commit hash, so rewriting a commit produces a new commit id while “the change ID generally remains the same” [81], and a change with more than one visible commit is called divergent. Conflicts are recorded in commits so that no command fails because of them, and rewriting a commit automatically rebases its descendants [80]. Mercurial’s changeset evolution records obsolescence markers, stored apart from changeset data, that say which changeset superseded which, and the wiki describes the result as “a history of changeset modifications that is orthogonal to the repository history of file modifications” [71]. Both designs let a developer say “the change I was working on yesterday” after five rebases and be understood, and both let two people rewrite the same region of history and have

the disagreement recorded, so the resolution is a decision somebody makes on purpose. sgt's treatment of two competing versions of one function is the same construction, and we take Jujutsu's divergent change and Mercurial's content-divergent changeset as evidence that recording the disagreement is the right response to it. We part company on what receives the durable identity. In both systems it is a commit, so the identity is exactly as coarse as the commit, and after a developer squashes a two-hour agent session into one commit, one change id covers all three requests and nothing smaller has a name. Peritext makes the same choice outside version control, giving formatting marks in collaborative rich text their own representation, because a record whose unit is the character gets the boundary of a bold span wrong when two people edit it at once [45, 50]. Figure 2 puts these records beside each other.

2.3 Version control that already knew about functions

Recording a version at the grain of a function has been built before, more than once, and the reason none of it displaced the commit is the reason we think it can now. Historage rewrites a Java repository so that every method sits in a file of its own, at which point git's existing machinery gives each method a history of its own [30], and FinerGit refines the file format so that git's rename detection keeps working when a method moves [36]. Stellation and Coven made the fine-grained unit the thing the repository stores rather than a transformation applied to it, so a developer could check out a single method and a team could see which methods their colleagues had open [12, 13]. MolhadoRef settled the problem Section 7.2 reports as our weakest link, by having the development environment record each refactoring as the developer performed it, so a rename entered the history as an operation instead of arriving as something a later tool had to guess at [18]. Mylyn recorded, for each task a developer worked on, the set of program elements they touched while working on it, and used that set to decide what the environment showed them [41]. All of this existed at least fifteen years before the first coding agent and all of it worked. Historage and FinerGit are in use today as instruments for mining software histories, which is research reading a repository from the outside, and a working programmer choosing what to run on Monday morning uses none of it.

We do not read that as a failure of engineering, because a finer grain buys a developer only what they cannot already supply for themselves, and until recently they could supply all of it. Whoever wanted the caching out of a 2010 repository had written the caching, so a system that told them which methods it touched was performing work they had already done, and the price of being told was a repository their colleagues could not clone with plain git. What has changed is not the value of the grain but the identity of the author, and a record that was redundant while the developer wrote the lines is the only surviving copy of the decomposition once they have stopped writing them. Mylyn's task context and sgt's named group of edits are the same object built for two different situations, and the difference is that Mylyn watched a developer's attention move through code that developer was writing, whereas the attention sgt reconstructs belonged to a process that has already exited.

The merge question in this literature was approached from the end opposite to ours. Horwitz, Prins and Reps showed that two versions of a program can be integrated automatically when a semantic analysis proves the changes do not interfere, and that it is the analysis which licenses the guarantee [37]. Apel et al.'s semistructured merge parses a file far enough to see that two edits landed in different methods, and resolves a large share of what a line-based merge reports as a conflict [1]. Lippe and van Oosterom argued for merging the operations rather than the states [49], Zeller and Snelting put the area on a formal footing by treating a version as a set with a logic over it, which is the construction Section 4.2 uses [89], and the distributed systems literature arrived at the same shape from another direction in the conflict-free replicated data types [65]. sgt goes the other way from the first two deliberately. Where semistructured merge combines two edits to one method as soon as it can see they touch different lines, sgt stops and asks a person, and Section 4.3 makes that argument where the rule is stated while Section 7.6 concedes that a repository with one person in it is the wrong place to have tested it.

2.4 Recovering a decomposition after the fact

Commits that address more than one concern distort the tools built from a history. Herzig and Zeller examined five open source Java projects, found such commits throughout them, and showed that separating those changes alters which files a defect prediction model identifies as the most likely to contain bugs [34, 35]. The untangling tools built after that finding partition a diff using definition and use relationships between the changed statements [5], use the fine-grained edit events an IDE can record [17], build a dependency graph across the versions a commit spans [61], or present a partition to the developer for correction [66, 69, 82], and they are evaluated on whether they reproduce a grouping that a human labelled. Two of those designs anticipate the move we make. Dias et al. read the edit events an IDE records while the developer is still typing, and Shen et al. put a partition in front of the developer and let them fix it, which is the arrangement Section 4.4 adopts. What none of them could assume is a statement of the boundary written down before the code existed, because in 2015 no such statement existed to be had, and the strength of their inference is a measure of how much they had to recover without one.

That is the loosened constraint, and it is worth being exact about what it buys us, because sgt performs the same inference they do. The grouping in Section 4.4 is a clustering over co-change and call structure, and it is no better than theirs; we would expect it to be worse, since it has to finish while a developer waits. What has changed is not the quality of the inference but what the inference is for. Their partition has to be right, because it is the answer. Ours has to be close enough that a name attaches to the right set, and the developer who reads a wrong name fixes it with a rename that moves no code.

sgt does depend on two results from this literature. Keeping a function's identity when it is renamed or moved is the problem that refactoring detection and tree differencing address [20, 21, 77], and it is the weakest link in our implementation, since a rename we fail to detect appears as one function removed and another added. Grouping edits into features is a clustering over which functions

change together and which call each other, and the co-change mining literature established both halves of that signal [24, 44, 85, 90]. We resolve the resulting graph with a community detection algorithm [75, 76]. Both are inferences, and Section 4 states what a developer does when they are wrong.

2.5 Histories the editor keeps

A second tradition puts the record in the editor and asks nothing of the developer. Azurite stores a programmer's editing operations at a finer grain than any commit and lets them undo one past edit while keeping the edits made after it [87, 88], and Yoon and Myers measured how often programmers undo and redo their way back to an earlier state, finding that they do it often enough for it to count as an ordinary part of editing [86]. Variolite lets a data scientist wrap a region of code as a variant and switch between variants in place, and Kery et al. built it after finding that people doing exploratory work avoid version control for exactly that work [42]. Related systems keep alternative program versions side by side [29, 70], replay a document's editing history [27], version at the grain of a single experiment [53], and help a person clean up a notebook after the fact [32, 43]. In all of these systems the record is a byproduct of working and the developer is never asked to prepare it, and sgt keeps that commitment. All of these systems stop at the boundary of one machine. The history lives in the editor's own store, so a colleague, a build, and a code review never see it, and the unit is a single edit event, so a set of related edits still has no name a person can use. sgt pays for crossing that boundary by writing its record into the git repository as files committed with the code, and Section 4 describes what that costs when a colleague who has not installed sgt commits to the same branch.

2.6 Layers over git built for the way people work now

The tools developers reach for today accept git's storage and add a layer above it. GitButler keeps several branches applied to one working directory at once, draws each as a vertical lane with its own staging area, and lets the developer drag a change into the lane it belongs to [22]. A developer should not have to decide which branch a piece of work belongs to before starting it, because that decision is easiest once the work exists, and we agree with GitButler completely on this. GitButler assigns an individual file change to a lane, so it cannot express the case at the centre of Figure 1, where three requests all edited `api.py::handle` and no assignment of `api.py` to one lane is correct. Graphite and GitHub's stacked pull requests take the ordered series as their unit. A stack is a sequence of changes in which each one targets the layer below it, and the tooling keeps the chain consistent, rebasing and retargeting the higher layers when a lower one merges and landing every unmerged layer beneath the topmost ready one in a single operation [23, 25]. A reviewer cannot read one large change with the care they can give to each of five small ones, and stacking is the right answer to that. Graphite's guide also states the precondition plainly, saying that developers must understand how to break down their work into small, incremental changes, and every command the tool offers operates on a decomposition the developer has already performed.

In an agent session that decomposition exists in the developer's request history and nowhere in the code.

The closest system to ours is sem, which parses about 32 languages with tree-sitter and reports which functions, methods and classes a commit changed, resolving an entity's identity across versions in three passes, by exact identifier, by a structural hash that ignores whitespace and comments, and by token overlap above 80% [2]. Its commands answer questions about entities. `sem impact` lists an entity's dependencies, dependents and tests, `sem log` surfaces which entities change most often and which change together, and `sem context` assembles a bundle about an entity for a model within a token budget. sem keeps all of this in a derived SQLite cache outside the repository, under the operating system's cache directory, and that placement buys sem three things. It makes sem safe to adopt and safe to abandon, gives entity-level answers about a history recorded years before sem existed, and requires agreement from nobody else on the team. The placement also bounds what sem can offer, because a record any teammate's ordinary commit can silently invalidate cannot be the record an operation acts on, and there is nowhere to write the outcome down. Every sem command therefore reports, and none of them changes state. A developer learns from `sem diff` that `get_cached` changed and then removes it with a line-level tool. sgt commits its record to the repository in order to act on it, and we pay for that in two places. A record that operations depend on has to be repaired when it is wrong, which Section 7.3 reports the current cost of, and everybody who pulls the repository now carries a record they did not ask for, which Section 7.5 works through from the point of view of the three people who never installed sgt. sem chose the safer of the two placements and we think we chose the more useful one, and the honest form of that sentence is that we have taken on an obligation sem does not have.

3 Five Things a Developer Cannot Ask For

The five scenarios below come from our own use of sgt while building it and from watching colleagues work with coding agents on repositories we maintain. Each one is an operation a developer tried to perform, stated in the words they used, and none of the five is available in git for the same underlying reason. In every case the developer named a set of edits and identified that set by the purpose the edits served, and the smallest thing git's record can name is a line inside a commit (Figure 2). Every scenario has a workaround and all the workarounds work, so our argument is about what they cost.

3.1 "Commit this when it is done, and I do not know yet when that is"

A developer who works through an agent still has to choose when to commit, and in this hour every moment available to them is wrong in a different way. Committing after each request writes a version of `api.py::handle` into the permanent history that existed for twenty-nine minutes and that nobody ever ran the test suite against, and it files the agent's rework of the cache under the retry commit, because the agent did that rework while adding the retry. Committing once at the end produces a single commit holding all three requests and the rename, and Figure 1 shows that

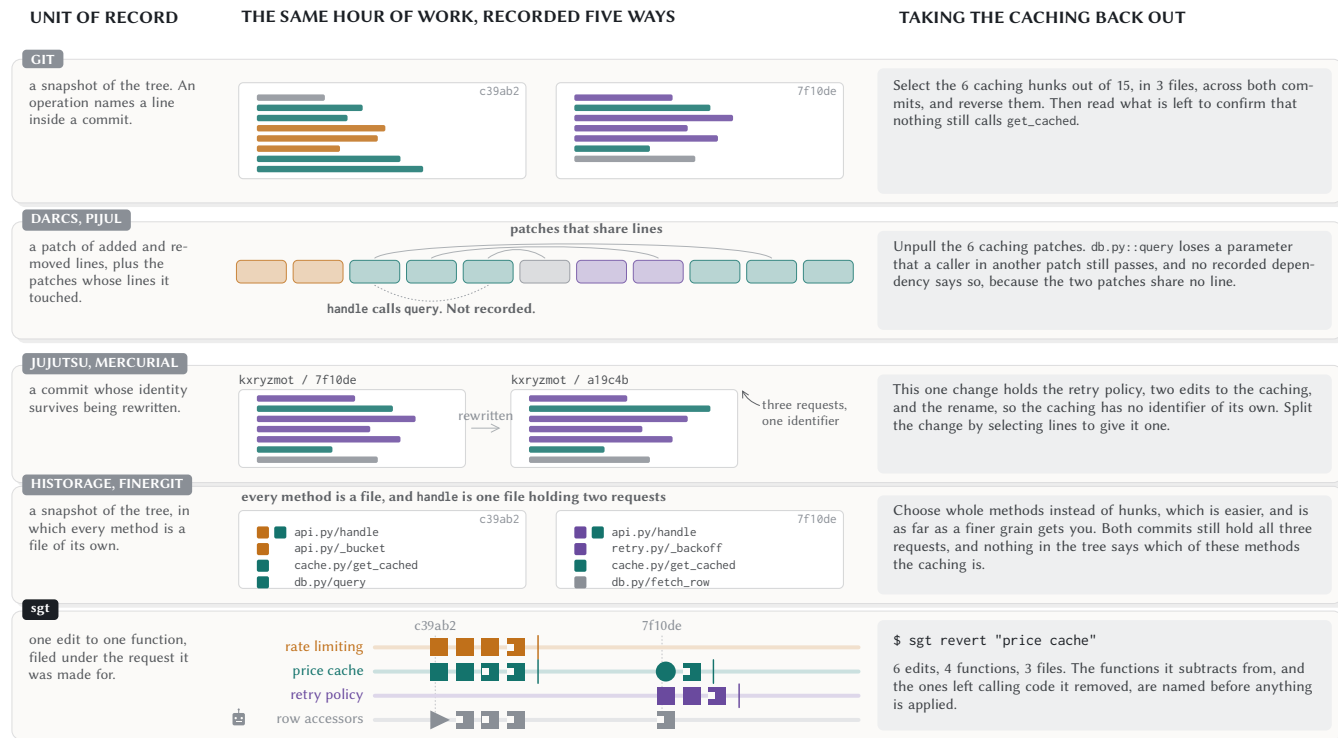


Figure 2: One hour of work, recorded five ways, with the same request taken back out of each. The fourth row already records at the grain of a method and still cannot answer the question, which is why we read the missing thing as the name of the request rather than the size of the unit.

state. Staging hunk by hunk produces four clean commits and asks the developer to decide, for each of fifteen hunks, which of those four purposes it serves. Green and Petre named these two costs premature commitment, where a notation forces a decision before the information the decision needs is available, and viscosity, the resistance a notation puts up once the decision turns out to be wrong [7, 26]. Git asks for the decomposition at the moment of writing. In a session driven by requests the boundary does exist at that moment and only the agent holds it, and the developer's cost of recovering it grows with every later request that touches the same functions.

The operation. Record the boundary the developer already stated, without asking them to state it a second time in a different notation.

3.2 "Change the rate limiter I built yesterday"

The rate limiting from 14:02 used a fixed one-minute window, and three days later it needs a sliding one. The work lands in two of the four functions the original request produced, and the developer's first question is what the rate limiting looked like before they touched it. Git can follow one function at a time. `git log -L :handle:api.py` follows a function through the history by matching a name pattern, and `git log -S` finds the commits where a given string appeared or disappeared. Both take one function or one string at a time, so the developer runs the first command four times, once per function they believe was involved, and assembles the four answers themselves. The set has no name, so nothing checks

whether four was the right number. Codoban et al. asked developers why they examine history and found the most common reason to be understanding the rationale behind a piece of code [15], and LaToza and Myers found questions about why code is the way it is to be among those developers report as hardest to answer [47]. The rationale in this case was three sentences of English, typed three days ago into a window that is now closed.

The operation. Name the work one request produced, see its current state and its previous state, and change it as a unit.

3.3 "Take the caching back out and leave the rest alone"

The caching turned out to serve a stale price for up to sixty seconds, and it has to go. Its six edits are spread over three files and two commits, mixed with nine hunks belonging to three other purposes. One of the six adds a parameter to `db.py::query` and another passes it from `api.py::handle`, and three functions the rename rewrote afterwards call a helper the caching introduced. `git revert` takes back a whole commit, so reverting either of these two removes work the developer is keeping. The developer checks out both commits, selects the six hunks by hand, reverses them, and then confirms that removing the parameter did not break `handle` and that the three functions the rename rewrote no longer call `get_cached`. The confirmation takes the time, because the record contains no statement that the `db.py::query` signature and the `handle` call site

were changed for the same reason. Brindescu et al. found that the difficulty of resolving a conflict is predictable from properties of the change, and that how far the change reaches is one of the properties that predicts trouble [8]. A hand-assembled reverse patch across two commits has exactly the properties their model warns about.

The operation. Remove a named set of edits, leave every other edit to the same files in place, and name the code that still depends on the removed set before anything is applied.

3.4 “Send just the export work to the release branch”

An export bug fix has to ship today, and it was written on a branch that also carries two unfinished features. `git cherry-pick` moves a commit, and the export work is two functions in a commit that also carries part of the unfinished retry policy. The usual answer is a temporary branch and an interactive rebase that splits that commit, and the split has to be done again if more export work arrives before the release. Darcs solves the part of this problem that is about lines, because a Darcs patch carries the dependencies it was recorded with and moving it involves no reapplication (Section 2). The export code calls a helper introduced by one of the unfinished features. No line-level record states that call, so the developer finds out when the release branch fails to import.

The operation. Carry a named set of edits and its dependencies to another branch, and be told before the operation which dependencies would come along.

3.5 “Let someone review the rate limiter while I keep building on it”

Rigby and Bird found that review practices across many projects converge on frequent review of small changes [62], and Bacchelli and Bird found that reviewers name understanding the change as their main difficulty [3]. Stacked pull requests exist so that a developer can keep each reviewed change small while the work continues on top of it. The rate limiter was finished at 14:30 and the caching that followed calls `api.py::_bucket`, so the two form a stack of two layers with the rate limiter underneath. Submitting them that way required the rate limiter to be on its own branch at 14:30, which required knowing at 14:30 that the caching would build on it. Graphite and GitHub’s stacked pull requests automate every step after that decision, including rebasing the upper layer when a reviewer asks for a change to the lower one (Section 2). The decision itself belongs to the developer and has to be made before the work exists, and in an agent session the layering becomes visible only once both layers are written. Church et al. call this a hidden dependency, a relationship the system relies on that the notation does not show, and they make the point about version control specifically [14].

The operation. Discover the layering after the work exists, and submit one layer while continuing to work on the layer above it.

3.6 What the five have in common

Every one of these five operations takes a set of edits as its argument, and the developer identifies that set by the purpose the edits served. Git expresses a set of lines, a set of files, and a set of commits,

and a developer can build all five operations out of those three. The price of building them is recovering a decomposition that nobody recorded, and that recovery stayed affordable for as long as the person doing it was the person who had written the code. Section 4 describes what we changed so that a developer can type the argument these five operations want.

4 What sgt Commits To

A version control system’s unit of record decides which questions a developer can put to their own history. `sgt` records one edit per changed function and files it under the request the edit was made for, and the commitments in this section are the consequences of that one choice, each stated with the price it carries. Figure 3 shows the first two at work on the sitting from Figure 1.

4.1 One edit to one function, filed under the request

A save reads the working files, compares them against the last recorded state, and writes one record per changed function. Each record names the function, the version of it the edit started from, the version it produced, and the other functions the new code refers to. The developer supplies one sentence, `sgt save -m "retry the upstream call when it returns a 5xx"`, and `sgt` reads everything else out of the files, so a record can be no more wrong than the code it was read from.

We chose the function as the unit because the line and the commit both fail on the same ordinary case. The price cache in Figure 3 added a parameter to `db.py::query` and passed a matching argument from `api.py::handle`. Those two changes share no line and no file. A record that bottoms out at a line has nowhere to write down that the two were made for one reason, so the developer who removes the cache a week later learns about the second one when the import fails. A record whose unit is the commit holds whatever was pending when the developer typed the command, which in Figure 1 is three requests and a rename the agent did on its own initiative, divided across two commits by the clock. A function is the smallest thing that both the developer and the agent name out loud while the work is happening, and the feature location literature works at that same unit for a related reason, since the code that serves one concern is a set of functions scattered across files [6, 19].

`sgt` finds functions in Python, TypeScript, and TSX with tree-sitter [9] and records every other file as its whole content, at the grain `git` already offers, so a repository whose interesting logic is in Go gets one record per file and none of the operations below becomes any sharper than the ones `git` offers today. Within the three parsed languages, a function can be smaller than a purpose. The rename in Figure 1 moved `db.py::fetch` to `fetch_row` and updated four call sites, and `sgt` writes five records for what the developer thinks of as one action. Those five records are one named group, so the mismatch shows up only when a developer reads the edit list underneath the name. A function can also be larger than a purpose, which happens when two requests change opposite ends of one fifty line function, and Section 4.3 describes what `sgt` does then.

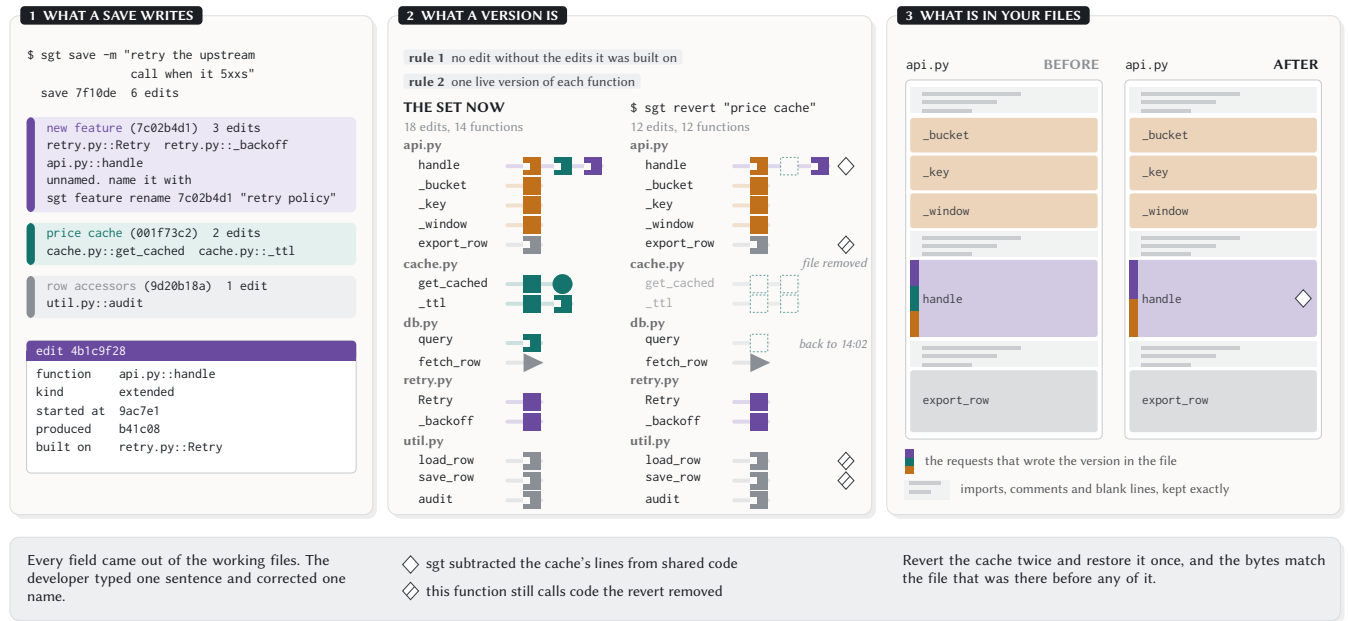


Figure 3: What one save writes down, what a version is, and where the bytes in a file come from. Every operation in this paper is arithmetic on the set in the middle panel, and the right panel is what that arithmetic does to a file the developer has open.

4.2 A version is a set of edits, and the files are rebuilt from it

Two rules decide which sets are versions. No edit is included without the edits it was built on, and each function has one included version. sgt rebuilds each file by writing out the included version of every function it contains and keeping the text between functions exactly as it was, which produces the bytes that are checked out. The middle panel of Figure 3 is one such set before and after a revert, and the right panel is the file each set produces.

Because a version is a set, revert, restore, diff, and switch all take a set and return a set, and two properties follow that a developer can check on their own repository. Reverting the price cache and then restoring it produces a file byte-identical to the one that was there before either command ran, because both operations are set arithmetic over the same records and neither reapplies a patch. Each of the four can also be shown in full before it happens, because working out the resulting files takes no more than rebuilding them from the proposed set. sgt revert "price cache" draws the twelve edits that would remain, names the functions whose text would change, and then asks apply this revert? [y/N]. Run with no terminal attached it prints the same preview and declines, so a revert nobody watched does not happen.

Three requests wrote `api.py::handle`, and in Figure 3 it carries an edit from the rate limiter, one from the cache, and one from the retry policy, in that order. Removing the cache's edit under the first rule alone means removing everything recorded above it in that function, and the retry policy is above it. We shipped that behaviour first. On a history where features interleave inside a shared function it removed a later feature in order to remove an earlier one, with a preview that counted edits and therefore looked reasonable, and we

changed the default in August 2026 after it emptied a test repository we were using for exactly this measurement. The default now keeps the later edits and subtracts the reverted feature's lines from the current text of the shared function with a three-way merge, the same operation git performs when it merges a text file. The version of `api.py::handle` that Figure 3 leaves in the file is produced that way, and the hollow diamond beside it in that figure is how sgt says so. The old behaviour is still available as `revert --take-dependents`, and its help text says what it removes.

That three-way merge is the only place sgt changes text that no person wrote, so sgt names every function it changes that way and every function it decided to leave alone. When the subtraction overlaps a line a later edit changed, sgt leaves the function exactly as it is and prints kept unchanged (the removal overlaps later edits—needs your edit) with the function named. When a function sgt is keeping still calls a function the revert removed, it prints still references removed code (fix or revert separately). Both of these are reports and neither stops the revert. sgt prints a warning and carries on wherever carrying on is possible. Running `sgt log --summary` in our own repository today prints 299 file(s) on disk differ from the recorded state—'sgt save' absorbs them, and every command still works while that is true. The cost of the default is that after the merge the shared function is text the developer has not read, and their test suite decides whether the revert was correct. We think that is the right place for the decision, and naming the function before the revert runs is as far as we are willing to go on the developer's behalf.

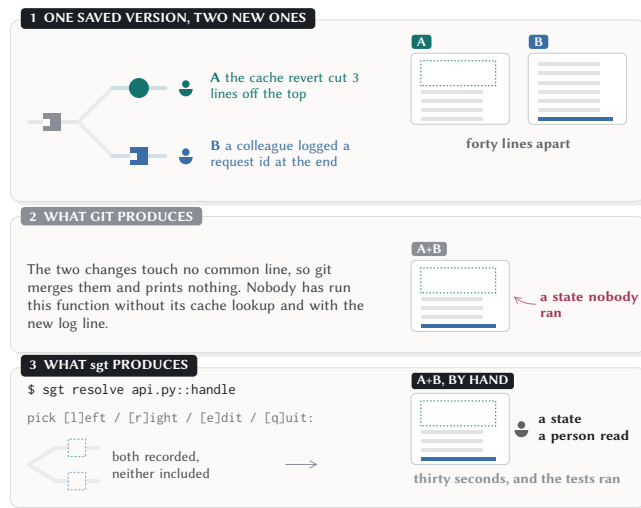


Figure 4: Two edits to one function from the same saved version, forty lines apart. Git produces a state nobody ran and sgt produces a question, and of everything sgt refuses this is the only refusal that can stop work which would have been correct.

4.3 Two versions of one function, and a person decides

When two edits change the same function from the same starting version, sgt records both, includes neither, and asks the developer. This is the commitment that costs the most goodwill in the first week, so we want to be precise about whose behaviour we are departing from and why theirs is right.

Git merges text and knows nothing about what a function is, and a system that has to work on every text file in every language anyone will ever write can afford to know nothing else. `git merge` is useful on a Makefile for that reason. In the case Figure 4 draws, two people had last saved the same version of `api.py::handle`. One of them reverted a caching feature, which took the cache lookup out of the first three lines of that function, and the other added a request identifier to the logging block at the end of it, forty lines away, with no line in common. Git merges the two without comment, and the function that comes out of the merge has never been run in that state by anybody. sgt sees one function with two versions from one parent, names it, and stops. `sgt resolve api.py::handle` prints the two versions beside each other and asks `pick [l]eft / [r]ight / [e]dit / [q]uit`, where the third choice opens the file in the developer's editor and the fourth leaves the fork open. Whatever they write has to pass the project's test command before sgt will include it. Where no test command is configured the developer supplies the verdict with `--override pass` and a `--reason`, and the reason and their name are recorded beside the resolution, so the next person to read that function finds out why it looks the way it does.

A developer pays for this by being stopped in cases where the merge would have been fine, and in one of those cases sgt stops

for nothing at all. Deleting a function and adding it back with byte-identical content in two places produces two versions from one parent, and sgt treats content that agrees exactly as a disagreement. Our position is that the cost of being asked is bounded and visible at the moment it is paid, and the cost of a silent textual merge inside one function is a failure that surfaces later somewhere else. We hold that loosely. A developer who merges thirty branches a day would reasonably weigh the two the other way, and for them the current design is worse.

4.4 The names are inferred, and correcting one moves labels only

Every figure in this paper labels a group of edits with a name like *price cache*, and those names are the part of sgt that guesses. Functions that changed in the same saves and that refer to one another are grouped together, using the change coupling signal established by Zimmermann et al. and Ying et al. [85, 90] and a community detection method over the resulting graph [76]. A language model then reads the saves in a group and writes the label, which is the descendant of a task attempted before language models existed, when Buse and Weimer generated a description of a change out of the change itself [10]. Membership is computed from the code and the label is written from the words, and those two steps fail in different ways, so we treat them separately.

We considered asking the developer to confirm the grouping at save time and decided against it, because answering that question takes the same knowledge commit hygiene takes, and a developer who could reliably answer it could also have staged the hunks. Shipman and Marshall collected the reasons people abandon systems that ask them to formalise their own work, premature structure among them, and the design they arrived at has the system propose a structure that its user may accept, modify, reject, or leave alone, in place of a system that demands one before the work is understood [67]. That is the arrangement here, and we are applying it under easier conditions than the ones they wrote about. The structure their systems tried to elicit was tacit, so a user asked to state it had first to work out what they thought. The structure sgt recovers was said out loud, in a request, before any of the code existed, and the only reason it needs recovering at all is that nothing wrote it down. What matters, then, is not whether the guess is right but what a wrong one costs, and correcting one moves labels and touches no recorded edit. `sgt feature rename 001f73c2 "price cache"` renames a group whose identifier stays what it was, `sgt feature regroup` moves functions between groups, and `sgt save --as "price cache"` files a new save into a group that already exists. Because identifiers survive all three, a revert written down in a code review comment last month still removes the same edits after somebody renames the group today.

Two failures show up often enough that a reader will hit them. A function that everything calls, an argument parser or a logging helper, changes in nearly every save and pulls otherwise unrelated work into one group, and the fix is a regroup the developer has to notice they need. A save with no message gives the language model less to read, and sgt says so at the time, printing `· no words captured` under the save it just wrote. The label on a group summarises what

the developer said about the work, so a developer who says nothing gets a name derived from the code alone.

4.5 What sgt will not do

sgt writes no code of its own. Every byte in the working files came from the developer or their agent, apart from the three-way merge splice in Section 4.2, which reports every case it cannot perform cleanly. We have been asked more than once during this work why sgt does not hand the shared function to a language model and have it redrafted after a revert, and our answer is that a version control system whose output nobody wrote is a version control system nobody can audit.

sgt also does not stand between a developer and git. The records are files under `.sgt/` committed alongside the code, `sgt git <args>` passes through to git untouched, and a colleague who has never installed sgt clones an ordinary repository with an ordinary history, and there is no server and no background process. Two of sgt's commands refuse to run. `switch` stops when there are unsaved edits, because sgt has nowhere to put them, and it names the files and prints record them with `'sgt save'`, or `commit / 'git restore'` those files, then re-run. `restore` stops when putting a set back would leave two versions of one function live, and there its message names the edits by identifier. The function whose two versions caused the refusal goes unnamed, which is a rough edge we have not fixed. Everything else sgt finds wrong it reports, and then it proceeds.

4.6 From the five operations to the commands a developer types

Figure 5 sets out the nine operations that act on the record, each with its cost beside it, and the five operations Section 3 ended on map onto them directly. The rest of what a developer types is housekeeping around those nine, `sgt init` to begin, `sgt log` and `sgt sync` to see where things stand, and the `sgt feature rename` of Section 4.4 to correct a name, and none of them changes which edits a version includes. Recording a boundary without declaring it is `sgt save -m`, run whenever the developer feels like running it, since the grouping is recovered afterwards from the code. Seeing the work one request produced is `sgt show "rate limiting"`, or `sgt show "rate limiting@1"` for one stretch of work on it alone. Removing a named set is `sgt revert "price cache"`, and carrying one to another branch is `sgt propose land --subset "export"`, which declines a subset that omits a group another chosen group depends on and names the group that is missing. Discovering the layering needs no command at all, because the cache edits record that they were built on `api.py::_bucket` and therefore on the rate limiter, so the developer has at any later moment the fact they needed at 14:30 and did not have. Section 5 runs these in order on one repository.

5 One Repository, Five Sittings

The repository here is a five thousand line Python service that answers price quotes for a trading dashboard, and the first sitting is the hour Figure 1 draws. We assembled the sequence from sessions we ran while building sgt so that one repository contains the five operations Section 3 asked for, in that order, followed by the one case where sgt stops and asks. Every command below runs and

every block of output came out of sgt as it is today. Output lines too wide for this column are wrapped, at the `·` separators where a line offers a list of alternatives, and are otherwise unchanged. Read the sequence for which decisions the developer is asked to make, and when in the work they arrive.

5.1 Monday, 14:02 to 15:20: two saves and four names

The developer made three requests in that hour, and the agent renamed a function on its own initiative. At 14:33, with the rate limiting finished and the price cache first drafted, the developer saved.

```
$ sgt save -m "rate limiting and a price cache on quotes"
✓ save c39ab2f "rate limiting and a price cache on quotes"
├─ new feature (4a1c9e02) — unnamed; name it:
│   sgt feature rename 4a1c9e02 "<label>"
│   api.py::_bucket, api.py::_key, api.py::_window +1 more
├─ new feature (001f73c2) — unnamed; name it: ...
│   cache.py::get_cached, cache.py::_ttl, db.py::query +1 more
├─ new feature (9d20b18a) — unnamed; name it: ...
│   db.py::fetch_row, api.py::export_row, util.py::load_row +1 more
△ one save touched 3 features — deliberate? `sgt log --map'
  shows them; `sgt feature regroup move' re-files work
✓ reverse this save: sgt undo
```

The developer typed one sentence and sgt wrote twelve records. The three groups have no names yet, because sgt writes records when the developer saves and works out the names when the developer next reads, and a name costs a language model call that has no business running while somebody is mid-session. The warning is the one line here that resembles what git asks for, and sgt printed it after the save was already written. The developer read it, agreed the sitting had been mixed, and carried on, and Section 3 opened with exactly that case.

The second save at 15:20 covered the retry policy and a rework of the cache the agent made while adding the retry. By then the first save's groups had names, so its echo listed `• Price Cache (001f73c2)` for the group that already existed and `◦ new feature (7c02b4d1)` for the retry work. `sgt log --tree` afterwards printed the four groups the hour produced.

```
Rate Limiting (4a1c9e02) · 4 symbol(s)
Price Cache (001f73c2) · 4 symbol(s)
Retry Policy (7c02b4d1) · 3 symbol(s)
Row Accessors (9d20b18a) · 5 symbol(s)
```

Three of the four names are what the developer would have written. The fourth is the agent's unrequested rename, named from the code because the developer never said anything about that part of the work, the failure Section 4.4 predicts. It is also the only one of the four whose name nobody needs, so nobody corrected it.

5.2 Thursday: changing the rate limiter

The fixed one minute window has to become a sliding one, and the developer starts from what the rate limiting is now.

```
$ sgt show "Rate Limiting"
```

before → after	COMMAND	WHAT THE DEVELOPER GETS	WHAT IT COSTS
WHAT WRITES RECORDS			
	<code>sgt save -m "..."</code>	One record per changed function, filed under a name for the request. The only command that adds to the record.	The grouping is a guess. Correcting it moves labels and changes no recorded edit.
WHAT ONLY READS THEM			
	<code>sgt show api.py::handle</code>	The request this function came from, the saves that touched it, and how much other work removing it takes along.	The answer is exactly as good as the grouping behind it.
	<code>sgt diff main feat/export</code>	The edits each version has that the other does not, in both directions.	It answers in functions, so seeing which lines moved still means opening the file.
	<code>sgt intent list</code>	Each feature cut into the stretches of work done on it, each with a name like <code>rate limiting@2</code> the other commands accept.	A language model places the cuts by reading the saves, so a stretch can start one save too early.
WHAT CHANGES WHICH RECORDS A VERSION INCLUDES			
	<code>sgt revert "price cache"</code>	The named set subtracted from the version, with every function it changes named before anything is applied.	A function later work also edited gets the set's lines subtracted from its current text, and the tests decide.
	<code>sgt restore "price cache"</code>	The same set back, byte for byte, with everything added since left alone.	Refused when putting the set back would leave two versions of one function live, and the refusal prints edit identifiers with no function name.
	<code>sgt switch feat/export</code>	The working files rebuilt from that branch's set.	Refused on unsaved edits, because sgt has nowhere to put them yet.
	<code>sgt resolve api.py::handle</code>	The two competing versions side by side, and whichever one the developer picks or writes is included once the project's test command passes.	Two edits at opposite ends of one function are also competing versions, and git would have merged them without asking.
	<code>sgt propose land --subset "export"</code>	One finished feature sent to the shared branch, with the unfinished one left behind.	Two functions that sit next to each other in one file can lose the blank line between them.

Figure 5: The nine operations that act on the record, grouped by what each one does to it. One command writes records and the other eight read them or change which of them a version includes, and no argument anywhere in the figure is a hash or a line number: every one of them is a request the developer named, a branch, or a function.

```
feature 4a1c9e02 "Rate Limiting"
4 edits · 4 symbols in 1 file · last touched 3d ago
symbols api.py::_bucket, api.py::_key, api.py::_window,
api.py::handle
saves c39ab2f rate limiting and a price cache on quotes
reverting this removes 4 edits
```

The four functions are the answer to the question that took four runs of `git log -L` in Section 3, and nothing here required the developer to guess that four was the right number of places to look. What this output does not say is that the price cache calls `api.py::_bucket` and therefore sits on top of the rate limiter, so a sliding window is a change underneath work that already depends on it. The record holds that fact, because the cache's edits name the functions they refer to, and `show` does not print it. The developer finds it out on Wednesday instead, when `propose land` refuses a subset for that reason, and a `show` that printed it here would have cost us one line.

The agent rewrites `api.py::_window` and `api.py::_bucket` and leaves the other two functions alone, and the developer saves, naming the group this time because they already know which one it is.

```
$ sgt save --as "Rate Limiting" -m "sliding window, 60s of buckets"
✓ save 1f0aee3 "sliding window, 60s of buckets"
└─ Rate Limiting (4a1c9e02) api.py::_bucket, api.py::_window
  ✓ named "Rate Limiting"
```

--as is the one place a developer states a grouping by hand, and it changed nothing here, because both functions were already in that group and the edits would have been filed there without it. They typed it because they knew the answer and had no reason to let `sgt` work it out. The rate limiting now has two stretches of work on it, and `sgt log --focus 4a1c9e02` names both.

```
• Rate Limiting 4a1c9e02 · 2 checkpoint(s)
[0■■■] 4a1c9e02@0 :rate-limiting Rate Limiting
[1■■■] 4a1c9e02@1 :sliding-window Sliding Window
operate:
sgt revert 4a1c9e02:sliding-window (one checkpoint, by name)
sgt revert 4a1c9e02 (whole feature)
sgt revert 4a1c9e02@1 (by index)
```

The developer never asked for these two stretches. `sgt` cut the work where their words changed, which in this repository is where one request ended and the next began, and either half is addressable on its own, so `sgt revert "Rate Limiting:sliding-window"` takes Thursday's work off and leaves Monday's four edits in place. The bar beside each name puts that checkpoint's edits in their place in the history, read left to right, with a middle dot where a span of the history holds none of them, so the terminal spends four characters on the encoding Figure 1 spends a panel on. The last three lines exist because we watched people on our own team discover that a stretch of work had a name and then stop, not knowing which command would accept it.

5.3 The following Monday: taking the cache out

The cache serves a stale price for up to sixty seconds and has to go.

```
$ sgt revert "Price Cache"
[the log region redrawn, with the 6 removed edits marked]
subtracted from shared code (later work kept): api.py::handle
removed going forward: cache.py::get_cached, cache.py::_ttl,
db.py::query
△ still references removed code (fix or revert separately):
api.py::export_row, util.py::load_row, util.py::save_row
apply this revert? [y/N]
```

The retry policy edited `api.py::handle` after the cache had edited it, so that function keeps the retry work and loses the cache's three lines, and a request made a week ago comes out from under a change made yesterday. Two functions existed only for the cache and come out with it, which empties `cache.py` and removes the file, and `db.py::query` loses the parameter the cache added and goes back to the text it had at 14:02. The three functions in the last line are row accessors the agent rewrote during its rename, and it pointed all three at `get_cached` while it was in there, so their edits belong to a request the developer is keeping and name a function the revert deletes. `sgt` names them and proceeds, because whether the repair is deleting three call sites or reverting the rename as well is not a question about the recorded edits.

The developer answered `y` and ran the test suite. It failed in the three places the report named and nowhere else, and the repair was deleting the cache lookup from each of them. `sgt` builds every one of these operations as set arithmetic so that a revert can be taken back exactly, and the developer tested that claim before trusting it.

```
$ sgt restore "Price Cache"
✓ restore applied — 0 edit(s) removed, 6 added.
('sgt undo' reverses this.)
```

Putting the cache back returned `cache.py`, `db.py::query`, and `api.py::handle` to the bytes they held before the revert, and it left the three call site repairs the developer had made in between alone, because a restore adds records to the set and rebuilds the files from the result without reapplying a patch. They had wanted the cache gone, so they reverted it a second time, and that run printed the same three consequence lines as the first.

5.4 Wednesday: shipping the export fix alone

A bulk export endpoint has to ship to the release branch today, and the branch it sits on also carries the retry policy, which is half finished. The export code goes into a new `export.py` and applies the rate limit before streaming, so `export.py::write_csv` calls `api.py::_bucket`. Nobody thought about that call as a dependency while writing it.

```
$ sgt propose create --base release
✓ proposal 3f9a2c81b04d (base release, +17 op(s), 4 feature(s))
$ sgt propose land 3f9a2c81b04d --subset "Quote Export"
× 'Quote Export' requires Rate Limiting -- include it in
--subset too
```

The developer asked to move one group and `sgt` named the group that has to come with it, the fourth operation Section 3 asked for. The record of the call to `api.py::_bucket` was written at the moment the export code was saved, and `sgt` reads it now to answer a question nobody was thinking about then. Git would have taken the cherry-pick and the release branch would have failed to import.

```
$ sgt propose land 3f9a2c81b04d --subset "Rate Limiting" "Quote
Export"
✓ propose land release: 8b21c0d4e7fa (+8 op(s))
```

Two groups went to the release branch in the order the records put them in, and the half finished retry policy and the accessor rename stayed behind. Those two groups are the stack from Section 3, assembled without the branch that had to exist before the work did.

5.5 Thursday: two versions of one function

A colleague on this repository had been editing `api.py::handle` as well, adding a request identifier to every outbound log line. The developer syncs.

```
$ sgt sync
△ sync origin/main: merged 4f2a1c8b93de
+7 op(s)
△ 1 OPEN FORK(S) — forked symbol(s) sit at the common
ancestor until resolved:
api.py::handle → sgt resolve api.py::handle
```

The remedy is printed as a function name because that is what the developer knows about the situation. Underneath it there is a command that takes the two edge identifiers directly, `sgt advanced merge-op 3c91ab02 8ee14d77`, and for the first several months of this system's life that was the line the fork warning printed, on the reasoning that it named the two things precisely. A developer who reads it learns that two records are in conflict and still has to work out which function they are in. Deriving the function's name and printing that instead is a one-line change we made late, and it is the clearest instance in this repository of a report that was accurate and useless. The two versions of `api.py::handle` are the one the cache revert produced, three lines shorter at the top where the cache lookup had been, and the colleague's, one line longer inside the logging block at the end. No line is common to the two changes, so git merges them without comment, the case Figure 4 draws. `sgt` prints them beside each other and asks.

```
— fork on api.py::handle — (◀ left = tip A ▶ right = tip B)
[the two versions of the function, side by side]
pick [l]eft / [r]ight / [e]dit / [q]uit:
```

The developer picked `e`, merged the two by hand in about thirty seconds, and `sgt` ran the project's test command before including the result. We are not going to claim the textual merge would have been wrong here. It would most likely have been right, and `sgt` charged thirty seconds and one interruption to establish that. We designed for the case where the merge is wrong, one edit adding an early return and the other edit's new code sitting below it, and the test run that would have caught it happening on a branch nobody has pushed yet. A developer who merges often and reads carefully is paying us for a certainty they already had.

5.6 What this cost

Across five sittings and one fork the developer made three decisions `sgt` could not make for them. They decided the export endpoint should ship without the retry policy, they decided how the two versions of `api.py::handle` should read, and they decided that the three functions still calling the cache should lose that call and keep the rest of the agent's rename. All three are decisions about the code itself, and none of the three is a question about which lines belong in which commit.

The developer paid for those three decisions in three places. One group of five edits carries a name derived from the code, because nobody said anything about that part of the work. The revert left `api.py::handle` holding text nobody had read, so their test suite decided whether the revert had worked. The fork stopped them on a merge that was probably fine. The second and third of those are direct consequences of Section 4.2 and Section 4.3, and on a repository this size we would choose both again. On a repository where twenty people edit the same functions each day, the fork rule is the first thing we would expect to have to change.

6 What Would Show This Is Wrong

Three claims in this paper fail independently of one another, and each one fails in a way that a different kind of evidence catches. The record either reproduces the bytes the developer's editor and agent left in the files or it does not, and that is a question about a program, settled by running it over histories that already exist. The grouping either lines up with the requests a developer made or it does not, and that is a question about an inference, settled by comparing it against sessions where somebody happened to write the requests down. A developer either gets more of these operations right with sgt than without it, or does not, and that is a question about people. The first has a procedure and a number we can print today, the second is under way on a corpus we are still assembling, and the third is designed and not yet run.

Not one number in this section comes from that third study. The measure that decides the argument is not the one a reader would expect us to lead with, and naming it before the data exists is the only way to be believed about it afterwards.

6.1 The rebuild has to be exact

If sgt reassembles a file into bytes that differ from what the developer left there, then every operation in Section 4 is set arithmetic over a fiction, and nothing else in this paper is worth reading. This is the one measurement whose failure would end the work rather than qualify it, so it goes first and it is the only one we insist on before adoption.

The procedure follows the shape of the claim. We walk a repository's history one commit at a time, mine each commit into records, rebuild every file from the set of records that commit's state implies, and compare the result against the blob git holds, byte for byte. A repository passes at a commit only if every file matches; a single differing byte in a single file is a failure at that commit, and we will report the fraction of files that rebuild rather than an average over repositories, because a developer cares whether their file comes back and not whether most files do. The corpus is open source Python and TypeScript repositories chosen for length of history rather than for anything else. We are still assembling it and have not run it, so the only number this procedure has produced so far comes from our own repository, which is the hardest case available to us because records written by five different versions of our miner sit in it and the version counter now stands at eight.

That number is fifty-six files that no valid set of records reproduces, which is one in seven of the files whose records name functions, and Section 7.3 gives that denominator and says what is in the fifty-six. What we do not know is whether fifty-six is the cost

of our own format changes or the cost of the design. Five miner versions leaving records in fifty-seven days is a rate no adopter will ever match, so the corpus run is how we intend to separate the two, since a repository sgt has never touched before cannot have our migrations in it.

6.2 Whether the grouping matches what was asked for

The names in every figure in this paper are inferred, and a developer who types `sgt revert "price cache"` is trusting that inference with the contents of their files. The question is not whether the clustering scores well against a partition somebody drew, which is how the untangling literature is evaluated for the good reason that its authors had nothing better to compare against (Section 2). The question is whether the set that comes back when a developer names one request is the set of edits that request produced.

That question needs a corpus where the requests were recorded, which agent sessions now supply as a matter of course, since the transcript of a session names each request and the files the agent touched while carrying it out. We mine each repository without reading any transcript, derive a reference grouping from the transcripts alone, and then ask two things of every request in the session. How much of what that request produced comes back when the developer names it, and how much else comes with it. Those two quantities are the two ways a revert disappoints the person who ran it. The first is the edit the revert missed and left sitting in the file, and the second is the edit belonging to a different request that the revert carried off with it. They fail asymmetrically, and the second is worse, because a developer who is handed too little notices when the code still runs the way it did and a developer who is handed too much has already lost work they meant to keep.

We expect the shared helper to be where this measurement goes wrong, because Section 4.4 predicts it and two months of daily use have not stopped it happening. An argument parser or a logging function that changes in nearly every save is coupled to everything by the signal the clustering reads, and it draws otherwise unrelated requests into one group. If most of the disagreement between our grouping and the transcripts turns out to sit on functions of that kind, then the fix is to weight a function's coupling down as the number of things it couples to grows, and that is a change to one term rather than to the design. If the disagreement is spread evenly across ordinary functions instead, then the coupling signal is not carrying the grouping and the developer should be asked at save time, which is the design we argued against in Section 4.4 and would have to argue for.

6.3 Three requests, a transcript, and code nobody in the room wrote

The study puts a developer in the position the paper is about, which means giving them a repository whose code they did not write and the three sentences that caused it. Each participant receives a five thousand line Python service, a transcript of the session that produced its most recent hour of work, and a request to take one of the three things in that transcript back out. This is the state of the developer in Section 3 a week later, and it is also the state of the developer who picks up a colleague's branch on a Monday, and it is

Table 1: The six measures of the study in Section 6.3, each with the result that would count against the design. The first row is the one we would report first, and the third row is the one a reader would expect us to report first.

What we measure	What we expect	What would count against us
Submissions the participant believed correct and that were not.	The gap here is wider than the gap in time. Removing a request touches code the participant never read, and in git nothing tells them so.	Equal rates in both conditions. The recovery we call expensive would then be something developers already do reliably, and Section 3 is a story about us.
Submissions that pass the check.	More of them with sgt, and the ones that fail fail in the places revert named out loud.	A failure in sgt that the preview did not name. A report that misses a consequence is worse than no report.
Time to the moment the participant says they are done.	Shorter with sgt, and we would not publish this alone.	Shorter with sgt while the first row is flat. sgt would then be a convenience, and we have argued for it as something else.
Lines of diff the participant opened before submitting.	Far fewer with sgt, since no command in Section 4 asks anyone to look at a hunk.	Participants reading the diff anyway. They would be checking sgt's report rather than using it, and after twenty minutes of that they are doing the git task with extra steps.
Forks sgt raised, and how many the participant judged pointless.	A small number, mostly judged pointless, all cheap.	Any participant switching the rule off, or a fork resolved by picking a side whose result then failed the check. Being stopped would have bought nothing.
Labels the participant corrected, and what the correction cost.	One or two renames, no regroup.	A regroup on the critical path of an operation. The inference would then be spending the developer's attention rather than saving it.

the only arrangement we found in which every participant knows exactly as much about the code as every other one. Participants who direct the agent themselves each remember a different amount of what came back, and that remembering is the thing whose absence we are claiming matters.

We recruit developers who use a coding agent as part of their ordinary work, because the difficulty we describe belongs to them and to nobody else, and we ask each of them to work through two repositories, one with git and one with sgt, in a counterbalanced order. In the git condition they use whatever they normally use, including a graphical client, since the argument is about what git's record can express and not about anybody's command line fluency. Each repository holds three requests whose edits interleave inside shared functions, and each has a different set of three so that a participant who has learned the shape of the first task cannot apply it to the second. The tasks are the three operations from Section 3 with a checkable outcome: take one named request out and leave the other two working, carry one request to a release branch on its own, and find the earlier state of a request in order to change it. We leave the other two scenarios out of the study because their outcome is a judgement about when to commit and how to stack, and we would be scoring participants against our preference rather than against a result.

A submission is correct when a script says so, and the script asks three questions. The tests belonging to the removed request fail, the tests belonging to the two kept requests pass, and no function outside the intended set differs from the reference repository. That third question is the one that catches the failure this paper is about, because a participant who removes the caching and quietly takes half the retry policy with it has produced a repository that passes its own tests.

Table 1 lists what we measure and what each measure would have to show for the design to be wrong. The first row is the one

that matters, and it is worth saying plainly why. A developer who spends an extra twenty minutes untangling a commit by hand has lost twenty minutes, and a developer who submits a repository they believe is clean and that still calls a function they deleted has lost something they will not find until the release branch fails to import. Hand untangling fails silently, and a design whose whole argument is that the developer no longer holds the decomposition has to be measured against silent failure rather than against the clock. We collect time because a reader would want it and because a tool that is correct and unbearably slow is not adopted, and we would not publish a time difference as the finding.

The last two rows of Table 1 measure the prices we stated in Section 4 rather than the benefits, and they are there because a study that measures only what a design buys is a study that cannot be wrong. A fork stops the developer and a wrong label makes them type a rename, both of which are countable inside the session, and both have a threshold past which we would change the design rather than defend it.

6.4 Fifty-seven days on one repository

Alongside those three, sgt has recorded its own construction, which is the only evidence any of this holds up past an afternoon. The system has been the version control for its own repository since it first ran, across fifty-seven days and three hundred and forty-six commits, so every commitment in Section 4 has been paid for daily, and two of the design decisions in this paper exist because a default we shipped removed work we wanted to keep. Fifty-seven days is enough to find a default that destroys work and not enough to find what a format decision costs after a year, and Section 7 reports that evidence, including a safety gate nobody configured and a coverage figure of twenty-five percent. It is the one part of the evaluation whose numbers are real today. We put it there rather than here because everything it has to say is a limit.

6.5 What none of this would tell us

Only the first of the three claims can be pushed past by running more of the same, since adding repositories to Section 6.1 sharpens that number and nothing else here improves with volume.

A recorded request is not an intention. A developer who types *make the price lookup faster* holds a view of what should change that the sentence does not contain, and the agent's response is a guess at the remainder. The alignment we measure against a transcript is therefore alignment with what was said, and the transcript sets a ceiling on it that no better clustering can pass. We think this is the right measurement anyway, on the ground that what was said is what the developer will type when they come back to remove it, but a reader who believes intention and utterance come apart more often than we do should read that number as an upper bound on something smaller.

Two hours with a transcript is not a week of forgetting. The cost we describe in Section 3 grows as a developer's memory of the session fades, and our design substitutes reading somebody else's transcript for that fading. A participant who has just read three sentences knows more about the session than the developer who typed them a week ago, so the git condition in our study is easier than the situation it stands for, and the difference we measure understates the one that matters. We prefer the understatement to a study nobody can run.

Finally, every task in the study has an answer a script can check, and we chose those three tasks for that reason. The operations a developer wants most may be the ones with no checkable answer, and deciding whether a stack of two reviews was the right shape is one of them. We can say what sgt makes possible there and we cannot measure it, and Section 7 keeps that distinction.

7 Where the Design Breaks

Every number in this section comes from one run of the command a user would run, on the repository that contains sgt, at a single commit. We took them together rather than as we needed them, because the repository is still being worked on and a set of counts gathered over a week is a set of counts of nothing. That repository holds fifty-seven days and three hundred and forty-six commits of daily use by the person who wrote the system, which is long enough to have found the limits below and not long enough to have found the ones that need a year, and we say which is which where we can tell.

7.1 Most of a repository is not functions

`sgt log --summary` prints 396 file(s), 6821 symbol(s), 154 feature(s), 25% entity coverage. We printed that last figure for months and it is the wrong figure, because its denominator is every file sgt has a record for and its numerator counts only the ones whose records name functions. Markdown holds no functions at any grain, so a repository with a lot of documentation in it reads as though sgt were failing at its one job.

Counting the records themselves says something different. This store holds records for 534 files, more than the 396 on disk today because a record outlives the file it was read from, and of those 534, 381 carry function-level records and 153 are recorded whole. The whole-file set is 82 Markdown files, 19 JSON, 15 $\text{\LaTeX}$, 13 HTML,

and a scattering of CSS, SVG, and configuration, and it contains no Python and no TypeScript. Every file in a language sgt parses has a function-level record. The design's coverage of code is complete and its coverage of a repository is a quarter, and those two sentences are about different things.

So the limit here is not the parser, and saying it was let us describe a property of software repositories as though it were a defect we could fix. Go and Rust have functions, a tree-sitter grammar plus a decision about which node kinds count as one is a day of work per language, and we would take both tomorrow. It would not move the quarter much. Three quarters of this repository is prose, configuration, and generated files, that ratio is ordinary, and a change to four lines of a deployment manifest has no function for the developer's request to be filed under no matter how many grammars we ship. Every operation in Section 4 still runs on those files, at git's grain, and none of them gets sharper than `git revert` would be. A reader deciding whether to adopt sgt should ask how much of the work they need to take apart happens inside functions, which is a different question from how much of their repository is code.

7.2 Where a function loses its history

Section 4.1 claims a record survives a rename, and that claim rests on a matcher with a threshold in it. sgt links a function across a commit first by its name, then by identical bytes, then by identical structure with formatting and comments ignored, and last by token overlap above 0.80, with a guard that rejects any pair whose token counts differ by more than a factor of two. The 0.80 comes from sem, which uses the same figure for the same job [2]. A rename that also rewrites about a fifth of the function falls below it and is recorded as one function removed and a different function added.

The developer then sees a function whose recorded history begins at that commit, with its earlier edits still filed under a group the function is no longer part of. An agent asked to clean up a module produces exactly this, renaming six functions and rewriting their bodies in one sitting. The detectors that get a function's origin right compare full syntax trees across the two versions [20, 24, 44, 77]. sgt uses the token count because it runs on every save and has to finish while the developer is waiting. Lowering the threshold would link functions that merely resemble each other, and a wrong link is the worse failure, because it files one function's edits under another function's name and gives the developer no reason to doubt the answer. `sgt advanced identity join` states the link by hand. Using it requires the developer to notice that a function's history restarted, and they installed sgt so that they would not have to notice things of that kind.

7.3 Fifty-six files we cannot rebuild

The same command reports 56 files whose bytes on disk no valid set of records can produce. The denominator is the 381 files whose records name functions and not the 534 the store holds, because a file recorded whole holds its own bytes and cannot fail to come back, so one function-level file in seven does not rebuild. sgt keeps every one of them, prints the count, names the first five, and points at `sgt advanced fsck --tree`. Nothing was deleted and nothing was quietly wrong, because sgt reports what it cannot do and then

carries on (Section 4.2). The developer loses the record for those files. A revert of a group that touched one of them leaves the file as it is, and `sgt show` on a function inside it answers from an incomplete chain.

We have not tracked down what put all 56 in that state. This store holds records written by five different versions of our miner in fifty-seven days, and some of these files are the residue of our own format changes. A developer adopting `sgt` on a repository with two years of history should expect a number of this kind on the first run, and should read it as work the design has not finished.

7.4 The gate nobody switched on

Section 4.3 says a resolved fork has to pass the project's test command before `sgt` will include it. The summary above also prints `oracle: unconfigured`. In the repository we use every day, resolving a fork means the developer picks a version and `sgt` takes their word for it, because nobody ever pointed `sgt` at our test runner.

A safety property a developer has to configure is a safety property most repositories will not have, so the strongest claim in Section 4.3 holds only where somebody did that setup, and the person who wrote the rule did not do it in the repository where the rule runs every day. The fix needs no research, since `sgt` could find the test command the project already uses and offer it during `sgt init`, and making the developer ask for it was the wrong default.

7.5 What a colleague sees

Because the records are files in the repository, everyone who pulls the repository gets them whether they asked for them or not, and this section works through what that means for three people who are not the developer running `sgt`: a colleague who has never installed it, a colleague who installed a different version of it, and the reviewer of a pull request.

The one we worried about most turned out to be the easy case. A colleague who has never heard of `sgt` commits ordinary code, and the next `sgt sync` mines their commits into function-level records on contact, the same way `sgt` mined the history that existed before it was installed. Their edits arrive at the right grain, they group with the work they belong to, and nothing has to be repaired. What does not arrive is the request, because there was no request to capture: their records carry no sentence, so the name for their group is drawn from their commit subjects, and where one subject accounts for most of a group `sgt` uses that subject as the name. A colleague who writes `fix tests` and `wip` gets a group named from the code instead, which is the same failure Section 4.4 describes for a save with no message. So the grain `sgt` recovers from anybody and the request only from somebody who was running it, and a team that half-adopts `sgt` gets a history that is half named.

The harder case is a colleague who does install it, and runs a different version. The identifier of an edit is computed from its content together with the version of the miner that read it, so two versions of `sgt` mint different identifiers for the same edit, and a sync across that gap would quietly alias records that mean different things. `sgt` refuses the whole union instead and prints what to do. That refusal is correct and it is also the cost we are least able to speak to, because Section 7.3 reports five miner versions inside fifty-seven days and every one of those transitions would have

stopped a collaborator. It stopped nobody, since there was nobody to stop, and a design decision that a solo repository charges nothing for is a decision that repository cannot evaluate.

The third case is the reviewer, and here we have no answer at all. A pull request that carries `sgt` records shows them as what they are on disk, a few hundred JSON files whose names are hashes, and no reviewer reads that. They review the code, as they should, and how anybody should read a large model-written change spanning many files is an open question being studied on its own [33]. But it means the record that makes revert work is not part of what anybody approved, so a resolution one developer reasoned through and a label another corrected both enter the shared branch unexamined. Section 4.3 argues that a fork should be settled by a person on the ground that a silent merge inside one function surfaces later as somebody else's bug. On a team, the person who settles it is still one person, and the review that would have caught them being wrong reads a diff their decision does not appear in.

7.6 Who should turn the fork rule off

This repository's history carries three git identities and one human being behind all of them, so every merge `sgt` has ever seen had the same person on both sides of it. That is the condition the fork rule was designed for and the only condition it has been tested under, and it is the condition in which the rule is most obviously right, because the person a fork stops is the person who wrote both versions and can therefore answer it in seconds. It is also the condition in which the rule is cheapest, and we should not draw a conclusion about teams from it. On a repository where twenty people edit the same functions every day, the same rule produces a queue of decisions whose two sides were written by people who have not spoken, and Nelson et al.'s account of how teams work through merge conflicts describes the barriers a queue of that kind meets [57]. We expect that team would switch the rule off and take the textual merge. Our evidence cannot tell them not to.

The setting we would build applies git's rule at the grain of a function, merging when the two edits changed no overlapping lines and asking when they did. Guimarães and Silva argue for telling a developer about a divergence while both sides are still being written [28], and that setting would let `sgt` do it without stopping anybody. We have not built it, because we cannot yet tell a reader which of the two should be the default, and the repository we would have tested it on has one person in it. This is the part of the design that a study of larger teams would most usefully overturn.

7.7 The record the agent already had

The grouping `sgt` recovers by clustering existed before any of the code did. The agent held it while it worked, and every agent we have used discards it when the session ends. A record written by the agent, one entry per request naming the functions it changed, would make the inference in Section 4.4 unnecessary, and it would fix the case in Section 5 where a rename nobody asked for got a name derived from the code.

Half of that record we do take, and the half we take is the half a vendor cannot spoil. Where the developer's agent is Claude Code, `sgt` installs a hook on prompt submission and stores every sentence the developer typed, with its time, so the words that named the work

are on disk before any code exists and `sgt` now answers with what was asked for rather than with a count of edits. That is twenty lines and one vendor's hook interface, and the same twenty lines have to be written again for the next agent, which is the price we decided we could pay, because what they capture is the developer's own English and a change of vendor cannot make yesterday's sentence wrong.

The half we do not take is the agent's account of which functions each request touched, and the reason is that a record of that kind is a claim about the files which we would have to either trust or check. Trusting it puts the correctness of a revert inside a transcript nobody validated against the code. Checking it means reading the files and computing the answer, at which point we did not need the account. What the agent's account would improve is the label, which is the part of `sgt` that is allowed to be wrong, and improving a label is worth a hint and not worth a dependency. So the split we have arrived at is that membership is read from code and names are read from words, and the words come from the hook where there is a hook and from the save message where there is not. What we do not know is whether an agent's account of its own edits is accurate enough to move membership too, and that is the measurement we would ask a reader of this paper to make before any other.

Open science

`sgt` is open source, and the repository it versions is the repository that implements it, so the artefact and the evidence in this section are the same object: a reader can clone it, check out the commit the artefact names, run `sgt log --summary`, and read every number in this section off the output. Run at a later commit the counts will have moved, since the repository is the working one. The study in Section 6.3 has not been run, and we will register its design, its measures, and the thresholds in Table 1 before recruiting, so that the first row remains the first row after we have seen the data. The two repositories the study uses, the transcripts that produced them, and the correctness script will be released with the results. No human-subjects data exists yet; the study will be submitted for ethics review before any participant is approached, and its only recorded data will be screen activity, task outcomes, and interview audio from consenting professional developers, with no personal data beyond what consent and compensation require.

References

- [1] Sven Apel, Jörg Liebig, Benjamin Brandl, Christian Lengauer, and Christian Kästner. 2011. Semistructured merge: rethinking merge in revision control systems. In *Proceedings of the 19th ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 190–200.
- [2] Ataraxy Labs. 2026. `sem`: Semantic version control built on Git. <https://github.com/ataraxy-labs/sem>. Accessed 2026.
- [3] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 712–721. doi:10.1109/ICSE.2013.6606617
- [4] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111. doi:10.1145/3586030
- [5] Mike Barnett, Christian Bird, João Brunet, and Shuvendu K. Lahiri. 2015. Helping developers help themselves: Automatic decomposition of code review changesets. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*. 134–144. doi:10.1109/ICSE.2015.35
- [6] Ted J. Biggerstaff, Bharat G. Mitbander, and Dallas Webster. 1993. The concept assignment problem in program understanding. In *Proceedings of the 15th International Conference on Software Engineering (ICSE)*. 482–498. doi:10.1109/ICSE.1993.346017
- [7] Alan F. Blackwell and Thomas R. G. Green. 2003. Notational systems: The Cognitive Dimensions of Notations framework. In *HCI Models, Theories, and Frameworks: Toward a Multidisciplinary Science*, John M. Carroll (Ed.). Morgan Kaufmann, 103–134.
- [8] Caius Brindescu, Iftekhar Ahmed, Rafael Leano, and Anita Sarma. 2020. Planning for untangling: Predicting the difficulty of merge conflicts. In *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 801–811. doi:10.1145/3377811.3380344
- [9] Max Brunsfeld and the tree-sitter contributors. 2026. `tree-sitter`: An incremental parsing system for programming tools. <https://tree-sitter.github.io/>. Accessed 2026.
- [10] Raymond P. L. Buse and Westley R. Weimer. 2010. Automatically documenting program changes. In *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 33–42. doi:10.1145/1858996.1859005
- [11] Ruijia Cheng, Ruotong Wang, Thomas Zimmermann, and Denae Ford. 2024. "It would work for me too": How online communities shape software developers' trust in AI-powered code generation tools. *ACM Transactions on Interactive Intelligent Systems* (2024). doi:10.1145/3651990
- [12] Mark C. Chu-Carroll and Sara Sprenkle. 2000. Coven: brewing better collaboration through software configuration management. In *Proceedings of the 8th ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE)*. 88–97.
- [13] Mark C. Chu-Carroll, James Wright, and David Shields. 2002. Supporting aggregation in fine grained software configuration management. In *Proceedings of the 10th ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE)*. 99–108.
- [14] Luke Church, Emma Söderberg, and Bhargava Elango. 2014. A case of computational thinking: The subtle effect of hidden dependencies on the user experience of version control. In *Proceedings of the 25th Annual Workshop of the Psychology of Programming Interest Group (PPIG)*.
- [15] Mihai Codoban, Sruti Srinivasa Ragavan, Danny Dig, and Brian Bailey. 2015. Software history under the lens: A study on why and how developers examine it. In *Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 1–10. doi:10.1109/ICSME.2015.7332446
- [16] Zheyuan (Kevin) Cui, Mert Demirel, Sonia Jaffe, Leon Musolf, Sida Peng, and Tobias Salz. 2025. The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. *SSRN Electronic Journal* (2025). doi:10.2139/ssrn.4945566
- [17] Martin Dias, Alberto Bacchelli, Georgios Gousios, Damien Cassou, and Stéphane Ducasse. 2015. Untangling fine-grained code changes. In *Proceedings of the 22nd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. 341–350. doi:10.1109/SANER.2015.7081844
- [18] Danny Dig, Kashif Manzoor, Ralph Johnson, and Tien N. Nguyen. 2007. Refactoring-aware configuration management for object-oriented programs. In *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. 427–436.
- [19] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature location in source code: A taxonomy and survey. *Journal of Software: Evolution and Process* 25, 1 (2013), 53–95. doi:10.1002/smr.567
- [20] Jean-Rémy Falleri, Flóreal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*. 313–324. doi:10.1145/2642937.2642982
- [21] Beat Fluri, Michael Würsch, Martin Pinzger, and Harald C. Gall. 2007. Change distilling: Tree differencing for fine-grained source code change extraction. *IEEE Transactions on Software Engineering* 33, 11 (2007), 725–743. doi:10.1109/TSE.2007.70731
- [22] GitButler. 2026. GitButler documentation: Parallel branches. <https://docs.gitbutler.com/features/virtual-branches/virtual-branches>. Accessed 2026.
- [23] GitHub. 2026. Stacked pull requests are now in public preview. <https://github.blog/changelog/2026-07-30-stacked-pull-requests-are-now-in-public-preview/>. Accessed 2026.
- [24] Michael W. Godfrey and Lijie Zou. 2005. Using origin analysis to detect merging and splitting of source code entities. *IEEE Transactions on Software Engineering* 31, 2 (2005), 166–181. doi:10.1109/TSE.2005.28
- [25] Graphite. 2026. Stacked diffs and how to use them. <https://graphite.com/guides/stacked-diffs>. Accessed 2026.
- [26] Thomas R. G. Green and Marian Petre. 1996. Usability analysis of visual programming environments: A 'cognitive dimensions' framework. *Journal of Visual Languages and Computing* 7, 2 (1996), 131–174. doi:10.1006/jvlc.1996.0009
- [27] Tovi Grossman, Justin Matejka, and George Fitzmaurice. 2010. Chronicle: Capture, exploration, and playback of document workflow histories. In *Proceedings of the 23rd Annual ACM Symposium on User Interface Software and Technology (UIST)*. 143–152. doi:10.1145/1866029.1866054
- [28] Mário Luis Guimarães and António Rito Silva. 2012. Improving early detection of software merge conflicts. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*. 342–352. doi:10.1109/ICSE.2012.6227180

- [29] Björn Hartmann, Loren Yu, Abel Allison, Yeonsoo Yang, and Scott R. Klemmer. 2008. Design as exploration: Creating interface alternatives through parallel authoring and runtime tuning. In *Proceedings of the 21st Annual ACM Symposium on User Interface Software and Technology (UIST)*. 91–100. doi:10.1145/1449715.1449732
- [30] Hideaki Hata, Osamu Mizuno, and Tohru Kikuno. 2011. Historage: fine-grained version control system for Java. In *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution (IWPSE-EVOL)*. 96–100.
- [31] Lile P. Hattori and Michele Lanza. 2008. On the nature of commits. In *Proceedings of the 23rd IEEE/ACM International Conference on Automated Software Engineering (ASE) Workshops*. 63–71. doi:10.1109/ASEW.2008.4686322
- [32] Andrew Head, Fred Hohman, Titus Barik, Steven M. Drucker, and Robert DeLine. 2019. Managing messes in computational notebooks. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–12. doi:10.1145/3290605.3300776
- [33] Lars Heander, Agnia Sergeyuk, Ilya Zakharov, et al. 2026. Trust-calibrated code review: A participatory design study of review workflows for LLM-generated multi-file changes. *arXiv preprint* (2026).
- [34] Kim Herzig, Sascha Just, and Andreas Zeller. 2016. The impact of tangled code changes on defect prediction models. *Empirical Software Engineering* 21, 2 (2016), 303–336. doi:10.1007/s10664-015-9376-6
- [35] Kim Herzig and Andreas Zeller. 2013. The impact of tangled code changes. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*. 121–130. doi:10.1109/MSR.2013.6624018
- [36] Yoshiki Higo, Shinpei Hayashi, and Shinji Kusumoto. 2020. On tracking Java methods with Git mechanisms. *Journal of Systems and Software* 165 (2020), 110571.
- [37] Susan Horwitz, Jan Prins, and Thomas Reps. 1989. Integrating noninterfering versions of programs. *ACM Transactions on Programming Languages and Systems* 11, 3 (1989), 345–387.
- [38] Daniel Jackson. 2021. *The Essence of Software: Why Concepts Matter for Great Design*. Princeton University Press.
- [39] Judah Jacobson. 2009. A formalization of Darcs patch theory using inverse semigroups. *Computational and Applied Mathematics Reports, University of California, Los Angeles* (2009). CAM report 09-83.
- [40] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues?. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- [41] Mik Kersten and Gail C. Murphy. 2006. Using task context to improve programmer productivity. In *Proceedings of the 14th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 1–11. doi:10.1145/1181775.1181777
- [42] Mary Beth Kery, Amber Horvath, and Brad A. Myers. 2017. Variolite: Supporting exploratory programming by data scientists. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 1265–1276. doi:10.1145/3025453.3025626
- [43] Mary Beth Kery, Marissa Radensky, Mahima Arya, Bonnie E. John, and Brad A. Myers. 2018. The story in the notebook: Exploratory data science using a literate programming tool. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–11. doi:10.1145/3173574.3173748
- [44] Sunghun Kim, Kai Pan, and E. James Whitehead. 2005. When functions change their names: Automatic detection of origin relationships. In *Proceedings of the 12th Working Conference on Reverse Engineering (WCRE)*. 143–152. doi:10.1109/WCRE.2005.33
- [45] Martin Kleppmann, Victor B. F. Gomes, Dominic P. Mulligan, and Alastair R. Beresford. 2019. Interleaving anomalies in collaborative text editors. In *Proceedings of the 6th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC)*. 1–7. doi:10.1145/3301419.3323972
- [46] Andrew J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks. *IEEE Transactions on Software Engineering* 32, 12 (2006), 971–987. doi:10.1109/TSE.2006.116
- [47] Thomas D. LaToza and Brad A. Myers. 2010. Hard-to-answer questions about code. In *Evaluation and Usability of Programming Languages and Tools (PLATEAU)*. 1–6. doi:10.1145/1937117.1937125
- [48] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A large-scale survey on the usability of AI programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1–13. doi:10.1145/3597503.3608128
- [49] Ernst Lippe and Norbert van Oosterom. 1992. Operation-based merging. In *Proceedings of the Fifth ACM SIGSOFT Symposium on Software Development Environments (SDE)*. 78–87.
- [50] Geoffrey Litt, Sarah Lim, Martin Kleppmann, and Peter van Hardenberg. 2022. Peritext: A CRDT for collaborative rich text editing. *Proceedings of the ACM on Human-Computer Interaction* 6, CSCW2 (2022), 1–29. doi:10.1145/3555644
- [51] Walid Maalej, Rebecca Tiarks, Tobias Roehm, and Rainer Koschke. 2014. On the comprehension of program comprehension. *ACM Transactions on Software Engineering and Methodology* 23, 4 (2014), 1–37. doi:10.1145/2622669
- [52] Pierre-Étienne Meunier and Florent Becker. 2024. The Pijul manual. <https://pijul.org/manual/>. Accessed 2026.
- [53] Hiroaki Mikami, Daisuke Sakamoto, and Takeo Igarashi. 2017. Micro-versioning tool to support experimentation in exploratory programming. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 6208–6219. doi:10.1145/3025453.3025597
- [54] Samuel Mimram and Cinzia Di Giusto. 2013. A categorical theory of patches. *Electronic Notes in Theoretical Computer Science* 298 (2013), 283–307. doi:10.1016/j.entcs.2013.09.018
- [55] Hussein Mozannar, Valerie Chen, Mohammed Alsobay, Subhro Das, Sebastian Zhao, Dennis Wei, Manish Nagireddy, Prasanna Sattigeri, Ameet Talwalkar, and David Sontag. 2024. The RealHumanEval: Evaluating large language models' abilities to support programmers. *arXiv preprint arXiv:2404.02806* (2024).
- [56] Peter Naur. 1985. Programming as Theory Building. *Microprocessing and Microprogramming* 15, 5 (1985), 253–261. Reprinted in *Computing: A Human Activity*, ACM Press, 1992.
- [57] Nicholas Nelson, Caius Brindescu, Shane McKee, Anita Sarma, and Danny Dig. 2019. The life-cycle of merge conflicts: Processes, barriers, and strategies. *Empirical Software Engineering* 24, 5 (2019), 2863–2906. doi:10.1007/s10664-018-9674-x
- [58] Santiago Perez De Rosso and Daniel Jackson. 2013. What's wrong with git? A conceptual design analysis. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward!)*. ACM, 37–52. doi:10.1145/2509578.2509584
- [59] Santiago Perez De Rosso and Daniel Jackson. 2016. Purposes, concepts, misfits, and a redesign of git. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. ACM, 292–310. doi:10.1145/2983990.2984018
- [60] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do users write more insecure code with AI assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2785–2799. doi:10.1145/3576915.3623157
- [61] Profir-Petru Pârțachi, Santanu Kumar Dash, Christoph Treude, and Earl T. Barr. 2020. Flexeme: Untangling commits using lexical flows. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 63–74. doi:10.1145/3368089.3409693
- [62] Peter C. Rigby and Christian Bird. 2013. Convergent contemporary software peer review practices. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 202–212. doi:10.1145/2491411.2491444
- [63] David Roundy. 2005. Darcs: Distributed version management in Haskell. In *Proceedings of the 2005 ACM SIGPLAN Workshop on Haskell*. 1–4. doi:10.1145/1088348.1088349
- [64] Advait Sarkar, Andrew D. Gordon, Carina Negreanu, Christian Poelitz, Sruti Srinivasa Ragavan, and Ben Zorn. 2022. What is it like to program with artificial intelligence?. In *Proceedings of the 33rd Annual Workshop of the Psychology of Programming Interest Group (PPIG)*.
- [65] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. Conflict-free replicated data types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*. 386–400. doi:10.1007/978-3-642-24550-3_29
- [66] Bo Shen, Wei Zhang, Christian Kästner, Haiyan Zhao, Zhao Wei, Guangtai Liang, and Zhi Jin. 2021. SmartCommit: A graph-based interactive assistant for activity-oriented commits. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 379–390. doi:10.1145/3468264.3468551
- [67] Frank M. Shipman and Catherine C. Marshall. 1999. Formality Considered Harmful: Experiences, Emerging Themes, and Directions on the Use of Formal Representations in Interactive Systems. *Computer Supported Cooperative Work* 8, 4 (1999), 333–352.
- [68] Jonathan Sillito, Gail C. Murphy, and Kris De Volder. 2008. Asking and answering questions during a programming change task. *IEEE Transactions on Software Engineering* 34, 4 (2008), 434–451. doi:10.1109/TSE.2008.26
- [69] Yida Tao and Sunghun Kim. 2015. Partitioning composite code changes to facilitate code review. In *Proceedings of the 12th Working Conference on Mining Software Repositories (MSR)*. 180–190. doi:10.1109/MSR.2015.24
- [70] Michael Terry, Elizabeth D. Mynatt, Kumiyo Nakakoji, and Yasuhiro Yamamoto. 2004. Variation in element and action: Supporting simultaneous development of alternative solutions. In *Proceedings of the 2004 CHI Conference on Human Factors in Computing Systems*. 711–718. doi:10.1145/985692.985782
- [71] The Mercurial contributors. 2026. Mercurial wiki: Changeset evolution. <https://wiki.mercurial-scm.org/ChangesetEvolution>. Accessed August 2026.
- [72] Harold Thimbleby. 1988. Delaying commitment. *IEEE Software* 5, 3 (1988), 78–86. doi:10.1109/52.2027

- [73] Yingchen Tian, Yuxia Zhang, Klaas-Jan Stol, Lin Jiang, and Hui Liu. 2022. What makes a good commit message?. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. 2389–2401. doi:10.1145/3510003.3510205
- [74] Linus Torvalds, Junio C. Hamano, and the Git contributors. 2026. Git: Distributed version control system. <https://git-scm.com/>. Accessed 2026.
- [75] V. A. Traag, P. Van Dooren, and Y. Nesterov. 2011. Narrow scope for resolution-limit-free community detection. *Physical Review E* 84 (2011), 016114. doi:10.1103/PhysRevE.84.016114
- [76] V. A. Traag, L. Waltman, and N. J. van Eck. 2019. From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports* 9 (2019), 5233. doi:10.1038/s41598-019-41695-z
- [77] Nikolaos Tsantalis, Matin Mansouri, Laleh M. Eshkevari, Davood Mazinanian, and Danny Dig. 2018. Accurate and efficient refactoring detection in commit history. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. 483–494. doi:10.1145/3180155.3180206
- [78] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. 1–7. doi:10.1145/3491101.3519665
- [79] Helena Vasconcelos, Gagan Bansal, Adam Fourney, Q. Vera Liao, and Jennifer Wortman Vaughan. 2025. Generation probabilities are not enough: Uncertainty highlighting in AI code completions. *ACM Transactions on Computer-Human Interaction* (2025). doi:10.1145/3702320
- [80] Martin von Zweigbergk and the Jujutsu contributors. 2026. Jujutsu documentation: Comparison with Git. <http://docs.jj-vcs.dev/latest/git-comparison/>. Accessed August 2026.
- [81] Martin von Zweigbergk and the Jujutsu contributors. 2026. Jujutsu documentation: Glossary. <http://docs.jj-vcs.dev/latest/glossary/>. Accessed August 2026.
- [82] Min Wang, Zeqi Lin, Yanzhen Zou, and Bing Xie. 2019. CoRA: Decomposing and describing tangled code changes for reviewer. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1050–1061. doi:10.1109/ASE.2019.00101
- [83] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. 2018. Measuring program comprehension: A large-scale field study with professionals. *IEEE Transactions on Software Engineering* 44, 10 (2018), 951–976. doi:10.1109/TSE.2017.2734091
- [84] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [85] Annie T. T. Ying, Gail C. Murphy, Raymond Ng, and Mark C. Chu-Carroll. 2004. Predicting source code changes by mining change history. *IEEE Transactions on Software Engineering* 30, 9 (2004), 574–586. doi:10.1109/TSE.2004.52
- [86] YoungSeok Yoon and Brad A. Myers. 2014. A longitudinal study of programmers' backtracking. In *Proceedings of the 2014 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 101–108. doi:10.1109/VLHCC.2014.6883030
- [87] YoungSeok Yoon and Brad A. Myers. 2015. Supporting selective undo in a code editor. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*. 223–233. doi:10.1109/ICSE.2015.43
- [88] YoungSeok Yoon, Brad A. Myers, and Sebon Koo. 2013. Visualization of fine-grained code change history. In *Proceedings of the 2013 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 119–126. doi:10.1109/VLHCC.2013.6645254
- [89] Andreas Zeller and Gregor Snelting. 1997. Unified versioning through feature logic. *ACM Transactions on Software Engineering and Methodology* 6, 4 (1997), 398–441.
- [90] Thomas Zimmermann, Peter Weißgerber, Stephan Diehl, and Andreas Zeller. 2004. Mining version histories to guide software changes. In *Proceedings of the 26th International Conference on Software Engineering (ICSE)*. 563–572. doi:10.1109/ICSE.2004.1317478